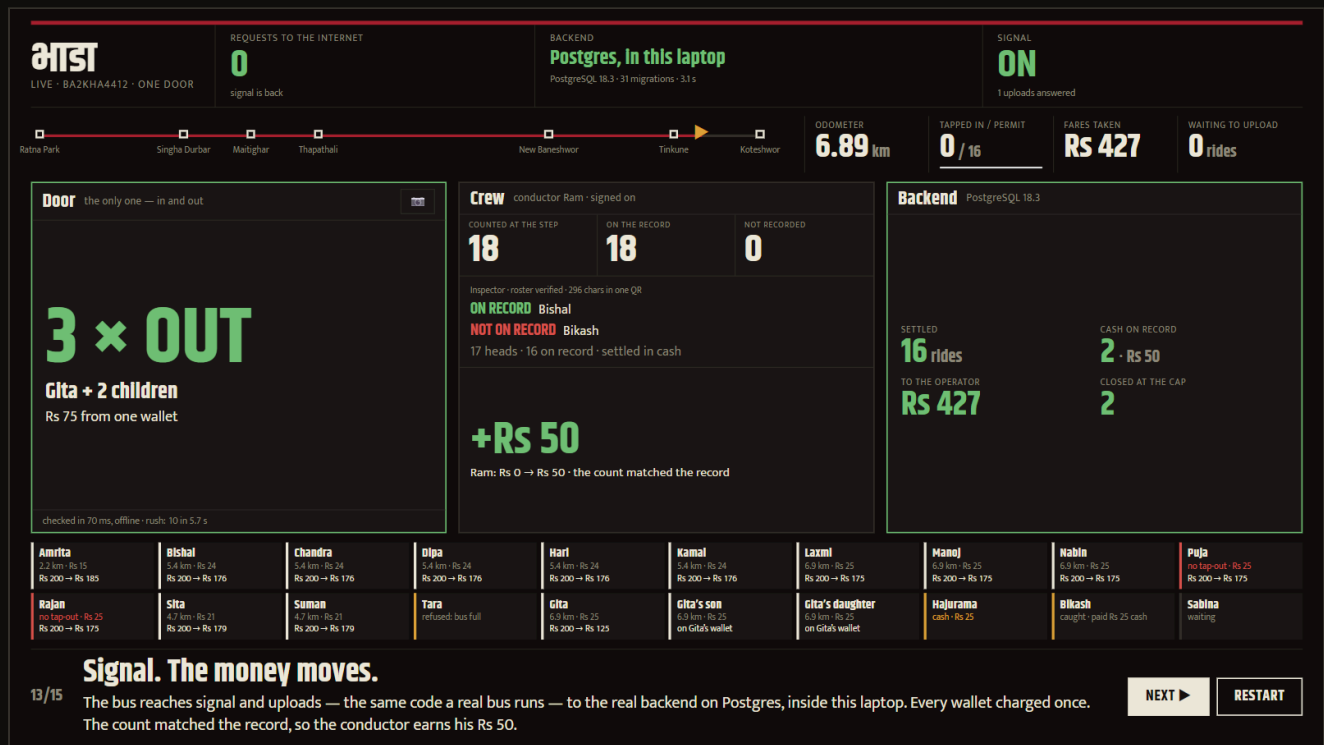


भाडा Bhada

Bus fares by the kilometre. Loads by the permit.
No signal needed for either.

Project documentation and demo guide — what the system is, how it works on a real one-door Nepali bus, what has been proved, and exactly how to demonstrate it to judges or a client.



LIVE SITE

bhada-one.vercel.app · demo at bhada-one.vercel.app/demo

VERSION

28 September 2026 · repository master, commit a365857 and later

STACK

React + Vite PWA · platform-free signing protocol · Supabase Postgres + Edge Function · Raspberry Pi validator

STATUS

Working software, proved by five proof suites; not yet run on a moving bus

Contents

1. The problem on Kathmandu's buses	9. How the data reaches the database
2. What Bhada is, in one page	10. How to give the demo — setup
3. A ride, end to end	11. How to give the demo — the fifteen scenes
4. Ground reality: every awkward case, answered	12. Questions judges ask, and the answers
5. The screens	13. What is proved, and what is live
6. Security: who can charge whom	14. Known limits and the road ahead
7. Money rules	15. Appendix: addresses and commands
8. The hardware: phones on day one, a Pi validator next	

0 requests to the internet needed to take a fare	~20 ms for the door to check a ride code, offline	482 named checks in the meter, legs and validator proofs	31 database migrations, every one run by the proofs
---	--	---	--

Bhada is a fare and occupancy system for the buses of the Kathmandu Valley. A passenger's phone signs a ride code; the bus's one door checks it with no network; the bus measures the road; the fare is priced by the kilometres actually ridden; and the money moves later, whenever any device on the bus reaches a signal.

It also answers the questions every earlier attempt left open: people who pay cash, families on one phone, people who ride without tapping, an inspector with no signal, a conductor who has to want it to work, and a bus with a single door that is both the entrance and the only exit.

1. The problem on Kathmandu's buses

Why a fare system here has to work differently

Kathmandu's buses have no fixed stops. People wave one down between junctions and step off wherever traffic stalls, yet fares are priced from a stage table. The conductor guesses the stage, the passenger argues, the cash comes back short, and nobody can prove anything.

The same buses leave Koteshwor carrying sixty people on a forty-two person permit, because nothing counts heads.

Three digital ticketing rollouts — Sajha, Bharatpur and Pokhara — worked in the demo and died on the bus, because each needed a mobile signal at the moment of payment. The stretch past Tinkune, the underpasses, a full bus of phones on one tower: that is where the signal goes, and that is exactly where the fare has to be taken.

What the system has to survive

REALITY	WHAT IT RULES OUT
No signal for minutes at a time	Any design that asks a server "is this passenger allowed?"
One door, in and out, 70 cm wide	Separate entry and exit gates; anything slower than about half a second per person
Many riders have no smartphone, or no balance, and pay in coins	A cashless-only system
Parents travel with children on one phone	"One phone, one person"
Some people will not tap	Trusting the tap count as the passenger count
The conductor's income today includes the skim	Any system the conductor has no reason to help
Cheap phones with wrong clocks	Strict time checks that refuse honest people

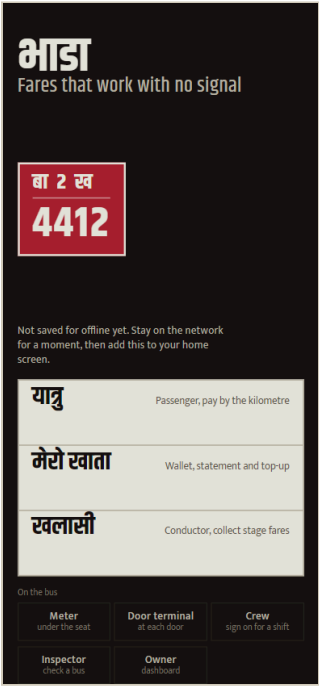
Design rule: **the fare is taken offline, always**. Reaching the internet is how money moves afterwards, never how a fare gets taken.

2. What Bhada is, in one page

Five parts, one protocol

PART	WHAT IT DOES	WHERE IT RUNS
The ride card	The passenger's own phone signs a fresh ride code (BT1) every 30 seconds, measures the ride with its own GPS as a witness, and keeps the signed receipt. One code can carry a whole family.	<code>/app</code> on any phone, installed as an app, works offline
The door	Checks each ride code's signature, opens or closes the ride, prices it by distance, signs the receipt (BM1), records cash fares (CT1), and refuses the next boarding once the bus is at its permit — while the door stays open for people getting off.	The conductor's phone (<code>/terminal</code>), or a fixed Raspberry Pi validator
The meter	Measures the road from GPS, keeps the count aboard, counts bodies through the door, signs the inspector's roster, and uploads everything when there is signal.	A spare phone under the seat (<code>/device</code>), or the Pi
The inspector	Checks the bus's signed roster and each passenger's code, offline, and counts heads against the record.	<code>/inspect</code> on the inspector's phone
The backend	Verifies every signature again, re-prices every fare from the published tariff, moves each rupee exactly once, pays the conductor's bonus, and gives the owner and the regulator their reports.	Supabase Postgres + one Edge Function; the owner dashboard at <code>/operator</code>

All five share one protocol library (`protocol/`) — the same file of signing and pricing code runs in the phone, the Pi, the backend and the proofs. A fare disagreement is settled by re-running the same arithmetic in all of them.



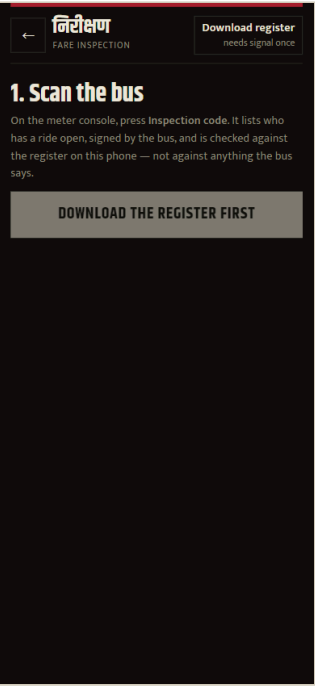
The app: passenger, account, conductor, and the bus-side tools



The ride card: a signed code, fresh every 30 s, and the tariff



The door terminal on the conductor's phone

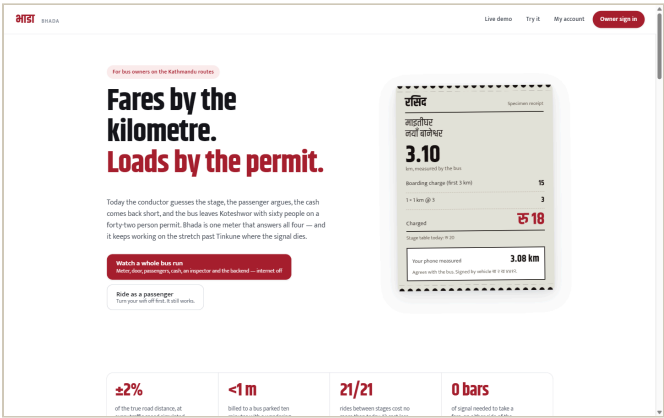


The inspector's phone

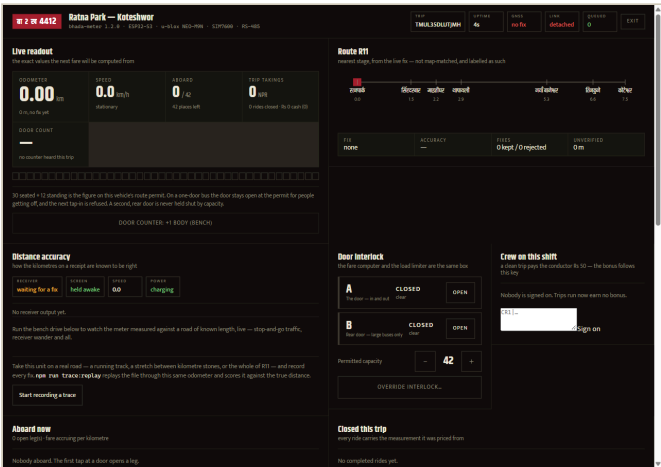
3. A ride, end to end

No step below needs a signal until the last one

- 1. **Board.** The passenger holds up the ride code. The door checks the signature against the key printed in the code, checks it is for this bus and less than five minutes old (either way, for phones with wrong clocks), and that its one-time number has not been used. It opens a ride stamped with the bus odometer. **Nothing is charged** — a fare quoted before anyone knows the distance is a guess.
- 2. **Ride.** The meter measures the road from GPS, filtered so a parked bus bills nothing (within $\pm 2\%$ in the proofs). The passenger's phone measures the same ride as a witness.
- 3. **Get off.** The same code, at the same door. A key with a ride open is getting off; any other key is getting on — one gesture, nobody chooses a button on a moving bus. The door subtracts the two odometer readings and prices the kilometres with the published tariff: **Rs 15 for the first 3 km, Rs 3 for each further km, never more than Rs 25** — so no ride costs more than today's stage fare.
- 4. **Receipt.** The door signs a BM1 receipt with the vehicle's key and speaks the fare in Nepali. The passenger's app compares it with its own measurement.
- 5. **Settle, later.** When the bus reaches any signal, the meter uploads the signed receipts with each passenger's signed tap beside them. The backend verifies both signatures, re-prices, and moves the money once. Sending the same upload again moves nothing.



The public site, bhada-one.vercel.app — every fare on it is computed by the same protocol code



The meter console: odometer, speed, aboard / permit, takings, door count, doors and crew

4. Ground reality: every awkward case, answered

What happens on a real, crowded, one-door bus

SITUATION	WHAT BHADA DOES	STATUS
Rush hour — 25 people in 15 s	A scanner reads a phone screen in about 0.2 s; the door decides in milliseconds. The code stays valid for five minutes, so people tap once they are inside and nobody blocks the step.	Built
One door, in and out	At the permit, the next tap-in is refused, but the door is never locked — it is the only way out. The regulator's record says "refused", not "locked".	Built
No phone, pays cash	The conductor records the fare in two taps: a CT1 cash ticket, signed by the bus, priced by the same tariff. The backend refuses one written down for less. It is what the crew owes the owner.	Built
No smartphone, regular rider	A Rs 20 NFC rider card, read by the door by its serial.	Built
A family on one phone	One code for up to five people. Each companion rides under a key derived from the payer's phone; every fare comes out of the payer's one wallet.	Built
Someone boards without tapping	A counter at the step counts bodies. Bodies minus (rides + cash tickets) is the leak, per trip, on the owner's dashboard. The conductor's Rs 50 bonus needs 90 % on the record, so the conductor is paid to make everyone tap or buy a ticket. A counter that goes quiet earns nothing.	Built
An inspector boards	The meter signs a roster of who has a ride open. The inspector's phone verifies it offline and marks each passenger ON RECORD or NOT ON RECORD, and counts heads against the record.	Built
Tap in, never tap out	The ride closes at the Rs 25 cap — the reason everyone taps out. A passenger whose phone died can claim the difference once, from their phone's own record.	Built
Zero balance	A metered ride may take a wallet to -Rs 50 rather than strand the passenger; the next top-up clears it.	Built
Screenshot shared	Every code carries a one-time number; a second use is refused at the door and again at the backend.	Built
Lost phone	Sign in on the new phone and move the money there; the old phone stops paying.	Live
No signal all day	Everything is kept on the device and uploaded in batches when signal returns, never twice.	Built

5. The screens

Two visual languages on purpose: ticket stock on the bus, a clean app in the office

The passenger



Ride code, the tariff, and "Just me"



"Me + 2": one code for three people, all on this wallet

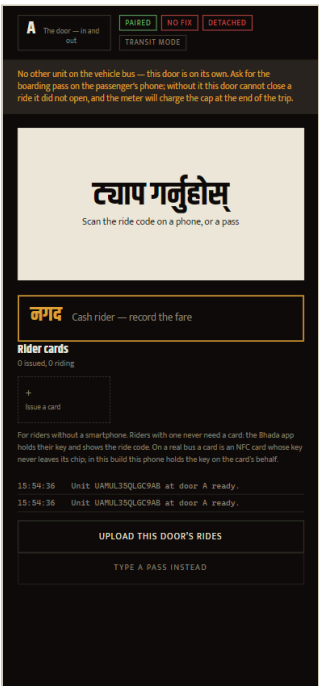


The account: balance, eSewa top-up, statement

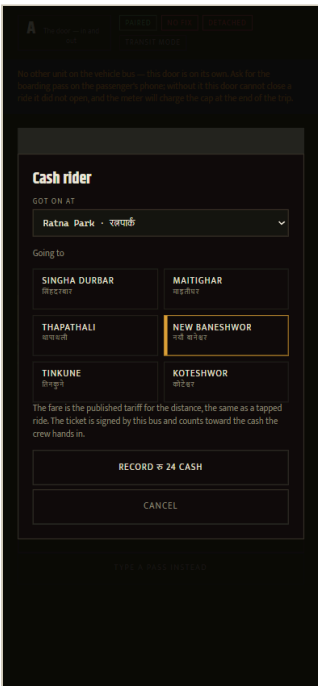


The inspector: download the register, then scan

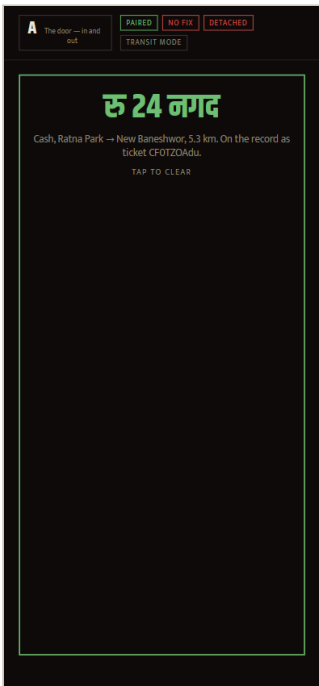
The conductor at the door



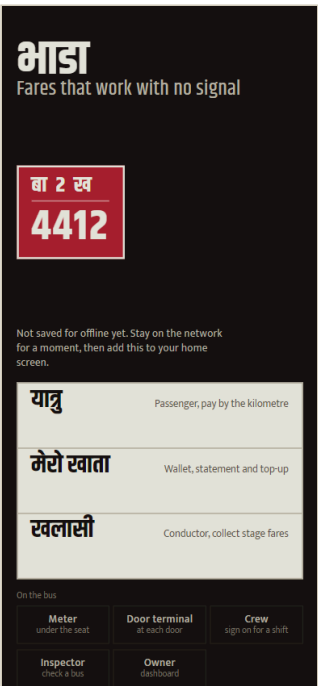
The door terminal: tap target, cash rider, rider cards



Cash rider: pick the stop; the tariff sets the fare



Recorded: Rs 24 cash, signed by the bus



Where each role starts

The owner

भाडा

OPERATOR

Sajha Yatayat

SAJHA - 2 buses

Sign out

कारोबार

Overview

बसहरू

Buses & routes

विवरण

DoTM return

कारोबार

Monday 28 September - all settled rides to date

उठेको

Collected

Rs 3842

यात्रु

Passengers

210

फेरा

Trips

2

बस

Buses

2

फिलोनितरमा

Metered rides, priced by the kilometre

यात्रा

Metered rides

105

यात्रु कि.मि.

Passenger-km

408

प्रति कि.मि.

per passenger-km

Rs 4.71

नासिपको

measured by odometer

98%

बा 2 रब 4412

BA 2 KHA 4412

61 rides - Rs 4.71 per km

बा 5 रब 2087

BA 5 KHA 2087

44 rides - Rs 4.71 per km

237

passenger-km

171

passenger-km

घण्टाअनुसार

व्यस्त खण्ड

2 rides were never tapped out and charged the route cap. A number that keeps growing points at a door where tapping out is being skipped.

The owner dashboard (/operator): takings, passengers, trips, passenger-km, Rs per km, share measured by odometer, per-bus figures, rides never tapped out. Shown here on sample data.

The meter

गा २ ख 4412

Ratna Park — Koteshtwor

bhada-meter 1.2.0 · ESP32-S3 · u-blox NEO-M9N · SIM7600 · RS-485

TRIP

TMUL3SDLUTJMH

UPTIME

4s

GNSS

no fix

LINK

detached

QUEUED

0

EXIT

Live readout

the exact values the next fare will be computed from

ODOMETER

0.00 km

0 m, no fix yet

SPEED

0.0 km/h

stationary

ABOARD

0 / 42

42 places left

TRIP TAKINGS

0 NPR

0 rides closed · Rs 0 cash (0)

DOOR COUNT

—

no counter heard this trip

30 seated + 12 standing is the figure on this vehicle's route permit. On a one-door bus the door stays open at the permit for people getting off, and the next tap-in is refused. A second, rear door is never held shut by capacity.

DOOR COUNTER: +1 BODY (BENCH)

Distance accuracy

how the kilometres on a receipt are known to be right

RECEIVER

waiting for a fix

SCREEN

held awake

SPEED

0.0

POWER

charging

No receiver output yet.

Run the bench drive below to watch the meter measured against a road of known length, live — stop-and-go traffic, receiver wander and all.

Take this unit on a real road — a running track, a stretch between kilometre stones, or the whole of R11 — and record every fix. `npm run trace:replay` replays the file through this same odometer and scores it against the true distance.

Start recording a trace

Aboard now

0 open leg(s) · fare accruing per kilometre

Nobody aboard. The first tap at a door opens a leg.

Route R11

nearest stage, from the live fix — not map-matched, and labelled as such

रत्नपार्क

सिद्धेश्वर

माझशीर

धापराली

नयाँ बानेश्वर

चिन्तपुरी

कोटेश्वर

FIX

none

ACCURACY

—

FIXES

0 kept / 0 rejected

UNVERIFIED

0 m

Door Interlock

the fare computer and the load limiter are the same box

A

The door — in and out

clear

CLOSED

OPEN

B

Rear door — large buses only

clear

CLOSED

OPEN

Permitted capacity

—

42

+

OVERRIDE INTERLOCK...

Crew on this shift

a clean trip pays the conductor Rs 50 — the bonus follows this key

Nobody is signed on. Trips run now earn no bonus.

CR1 | ...

Sign on

Closed this trip

every ride carries the measurement it was priced from

No completed rides yet.

The meter console (/device): live odometer, occupancy against the permit, takings with cash, the door count, the interlock, crew sign-on, and the inspection code.

6. Security: who can charge whom

Every message that matters is signed; the backend re-checks all of it

CODE	SIGNED BY	WHAT IT PROVES
BT1	The passenger's phone	"This key is boarding this bus, now." Five-minute window, one-time number. Without it no ride can be billed — a bus's key alone cannot charge anyone.
BG1	— (a container)	Up to five BT1 codes in one QR, for a family. Nothing new is signed.
BO1	The vehicle	The boarding pass: where and when a ride opened, so a second door can close it with no network.
BM1	The vehicle	The receipt: distance, fare and tariff. Re-priced by the backend; a mismatch is refused.
CT1	The vehicle	A cash fare at the tariff for the declared stops. Recorded, never charged to a wallet.
RS1	The vehicle	The inspector's roster: open rides, cash tickets, the door count. Good for three minutes.
PK1	Wallet and day-key	Links today's key (and a family's companion keys) to the wallet. Operators see a different key every day, so they cannot follow a passenger's commute.
CR1	The conductor	Sign-on for a shift, so the bonus goes to the person who worked it.
AL1	The wallet	Links a phone's wallet to a login, so a lost phone does not lose the money.
BD1 / AT1	Passenger / an issuing office	A dead-phone claim against a ride charged the cap; a student or senior concession card.

Attacks that are refused, in the proofs

- A screenshot of somebody's code — refused as already used, at the door and in the database.
- A code forged with somebody else's key; a code for another bus; an expired code.
- A receipt with no passenger consent behind it; one genuine tap stretched over a ride never taken.
- A receipt or cash ticket priced differently from the published tariff.
- A stranger re-keying a public validator; an inspection roster edited after signing.
- Replaying an entire upload — every item comes back "replay" and no rupee moves.
- The public anon key reading owners' data — every table and view is checked in the proofs.

7. Money rules

The numbers that are policy, each held by a proof

RULE	VALUE	WHERE IT LIVES
First 3 km, including boarding	Rs 15	TARIFF , protocol/meter.mjs
Each further kilometre	Rs 3	same
Most any ride can cost (today's end-to-end stage fare)	Rs 25	same; checked over all 21 stage pairs
Ride never tapped out	Rs 25 (the cap), one claim back	protocol/dispute.mjs
Ride code valid for	300 s either way	door and SQL, kept equal
Wallet may go below zero on a metered ride	-Rs 50	policy.mjs and SQL
Conductor's clean-trip bonus	Rs 50, once per trip	policy.mjs; paid from the operator's payable, not minted
...needs at least	5 rides on the record	cleanTripVerdict()
...and, with a door counter, on the record	90 % of bodies counted	cleanTripVerdict()
...and no pulled plug, no interlock override, no impossible rides	—	cleanTripVerdict()
Top-up	eSewa (gateway + manual)	payments function

Money moves on the wallet behind a key, never on the key a ticket names: day-keys and companion keys have no money of their own. Every rupee moves inside one of a handful of SQL functions, only after the Edge Function has verified every signature, and every repeat is answered "replay" rather than charged.

8. The hardware

Day one needs nothing new; a fixed validator comes next

Day one: the phones the crew already carries

UNIT	DEVICE	APPROX. COST
Meter	A spare Android phone under the seat, on the bus's 12 V	Existing old phone
Door	The conductor's own phone, with an OTG USB QR scanner (GM65-class) for 0.2 s reads	about ₹1,500–2,500 for the scanner
Rider cards	NTAG213 cards, read by the door phone's NFC	about ₹15–20 each
Passengers	Their own phones; cash riders need nothing	—

Next: the Bhada door validator (Raspberry Pi)

A box beside the door that runs the **same door code as the phone, unchanged**: a Pi Zero 2 W, a GM65/GM805 QR scanner, a PN532 card reader, a 2.8" screen, a DS3231 clock, an RGB light and a buzzer, powered from the bus through a protected vehicle DC-DC converter — no battery, no separate GPS. Roughly ₹4,500–7,000 in parts at Indian retail; check local prices. It refuses to validate if it cannot trust its clock, and its public-facing scanner can only load a vehicle key while it has none. **No pin drives a door.**



The validator's screens as it draws them on its 320×240 display. Every QR code shown is decoded again from the pixels by the validator proof. The validator is proved in software and has not yet been run on its parts.

The door counter

A break-beam sensor at the step (a few hundred rupees) wired to the meter or validator, pulsing once per body. In the software today it is the meter's `countBoarding()`; the physical beam is the next piece of hardware to fit.

9. How the data reaches the database

Store on the bus, forward when there is signal, never twice

QUESTION	ANSWER
Where is a ride kept?	On the device that decided it: IndexedDB on a phone, SQLite on the Pi — written to disk before the door says ✓.
Which protocol?	HTTPS POST of JSON to the sync Edge Function. The same batch format from every device. No MQTT broker, no extra server.
How does the bus get internet?	Any link at all: the phones' own data, a 4G USB dongle on the Pi, the conductor's hotspot, or depot Wi-Fi at the terminus.
What if the upload dies half way?	Every item is idempotent: a second copy is answered "replay", never charged. The queue is cleared only for items the backend has ruled on.
Offline for a week?	Everything stays queued and goes up 150 items at a time; retries back off from one minute to ten.
How is it authenticated?	By signatures, not passwords: the vehicle's on each receipt and cash ticket, the passenger's on each tap. The vehicle key is frozen on first sight.
How does the owner know a bus is alive?	Every upload is a heartbeat; the dashboard shows which bus stopped reporting.

Release order is always: database migrations, then the sync function, then the site — so the backend always understands what the phones send.

10. How to give the demo — setup

Five minutes, one laptop, a projector, no bus, no internet

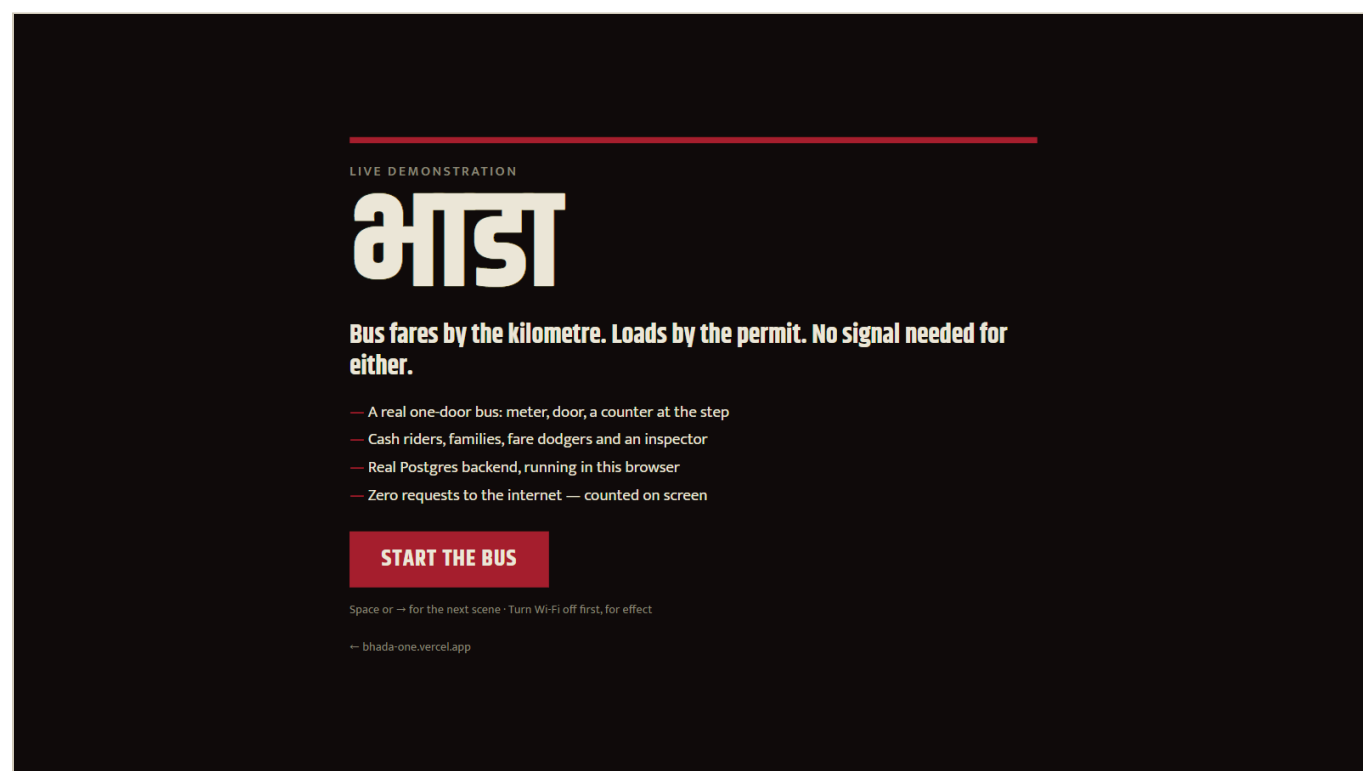
The demo is at `/demo`. It is not a video or a mock-up: the real meter, the real door, the real upload code and the real backend — every database migration running on Postgres inside the browser — on one screen. The header counts every request that tries to leave the laptop. It stays at zero.

The day before

- ☐ Clone the repository on the demo laptop and run `npm install`.
- ☐ Run `npm run build` then `npm run serve`, and open `http://localhost:4173/demo`. The laptop serves itself, so airplane mode cannot break it.
- ☐ Click through all fifteen scenes once, end to end (about four minutes).
- ☐ Record the run with the screen recorder as a backup video.
- ☐ Install `bhada-one.vercel.app/app` on your own phone (Add to home screen) for the "your turn" scene; check the laptop camera works in the browser.
- ☐ Charge everything. Bring a clicker: `→` or the space bar moves to the next scene.

On stage

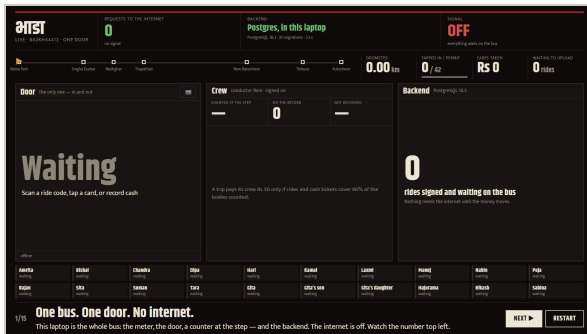
- ☐ Open the demo, full screen (F11). Turn Wi-Fi off in front of the audience.
- ☐ Press **Start the bus**. Postgres starts in the browser in about three seconds.
- ☐ Say the one sentence for each scene (next pages). Let the screen do the rest.
- ☐ If anything fails: do not debug on stage. Play the backup video.



The demo's opening screen

11. The fifteen scenes

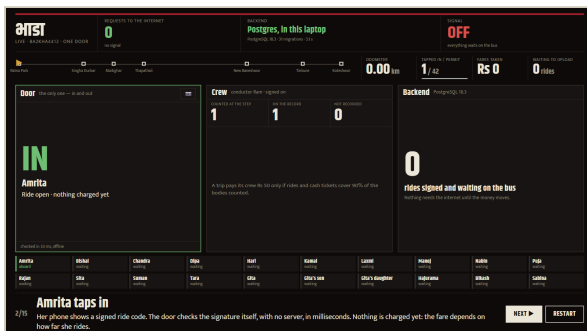
What happens, and what to say — press → for each



1 One bus. One door. No internet.

The laptop is the whole bus: the meter, the door, a counter at the step, and the backend.

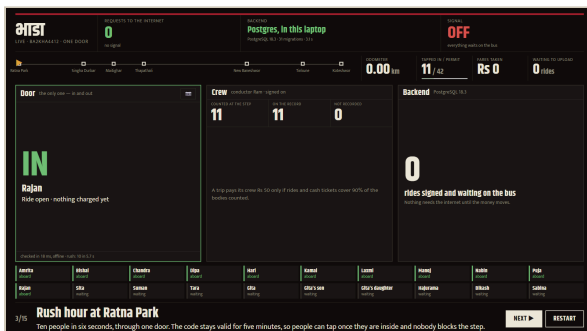
"The internet is off. Watch the number top left — it stays at zero."



2 Amrita taps in

The door checks her signed ride code itself, in milliseconds. Nothing is charged yet.

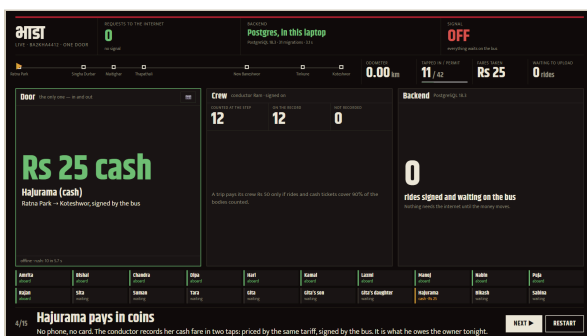
"The fare depends on how far she rides, so nobody guesses it."



3 Rush hour

Ten people in about six seconds through one door.

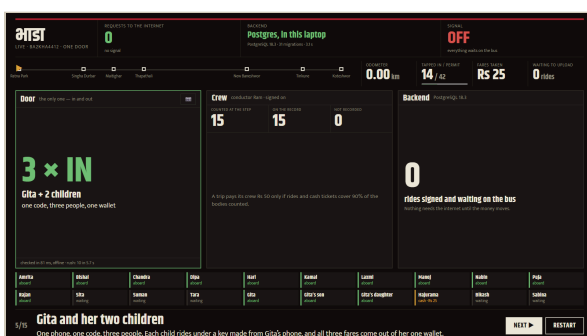
"The code is good for five minutes, so people tap once they are inside. Nobody blocks the step."



4 Hajurama pays in coins

The conductor records her cash fare: a signed cash ticket at the tariff.

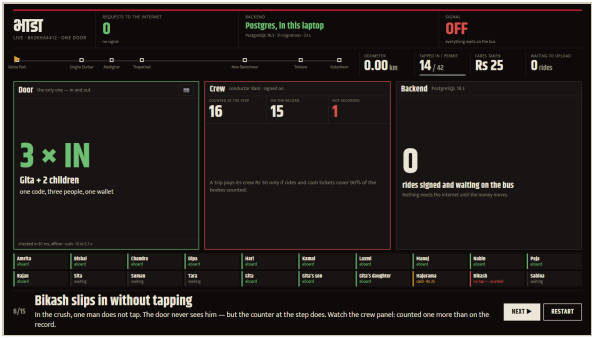
"No phone, no problem. That cash is now on the record — it is what the conductor owes the owner tonight."



5 Gita and her two children

One code boards three people.

"One phone, one wallet, three rides."



6 Bikash slips in without tapping

The crew panel shows 16 counted at the step, 15 on the record.

"The door never saw him. The counter at the step did."



7 The bus moves

The meter's GPS odometer along the real R11 corridor, stop-and-go.

"Every fare on this bus is priced against this one odometer."



8 Three ways to cheat

A screenshot of Sita's code, a forged code, a code for another bus: all refused.

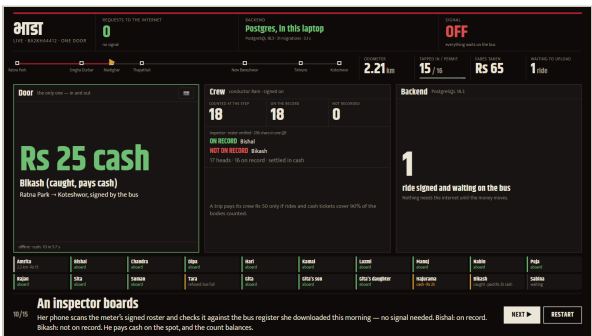
"Sharing a screenshot does not work, and neither does forging one."



9 The permit, not the conductor

The bus fills to its permit; the next person is refused; Amrita still gets off through the same door.

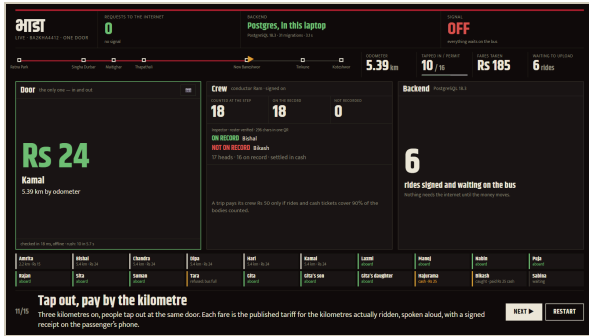
"The count decides, not anyone's mood. And the only door is never locked."



10 An inspector boards

The signed roster is verified offline. Bishal: on record. Bikash: not. He pays cash, and the count balances.

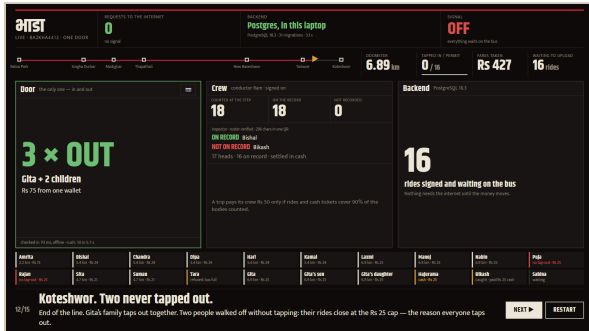
"No signal needed to inspect a bus, and the bus cannot lie about who is on it."



11 Tap out, pay by the kilometre

Fares by the kilometres actually ridden, spoken aloud.

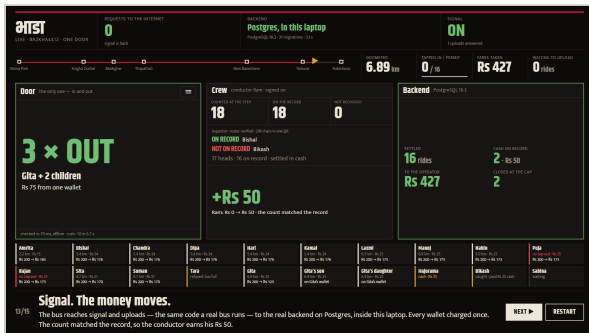
"Three kilometres, fifteen rupees. Five kilometres, twenty-one. Never more than today's stage fare."



12 Koteshwor — two never tapped out

Gita's family taps out together; two people who walked off are charged the Rs 25 cap.

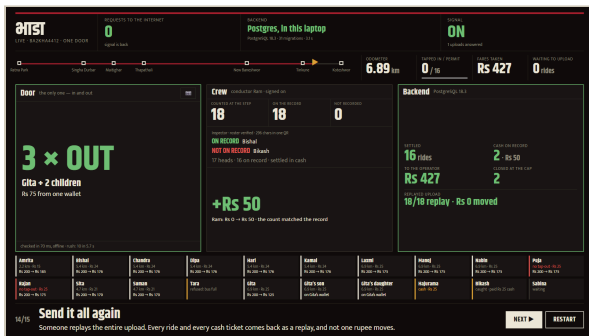
"That cap is why everyone taps out."



13 Signal. The money moves.

Real settlement on Postgres: every wallet before → after; the conductor earns Rs 50 because the count matched the record.

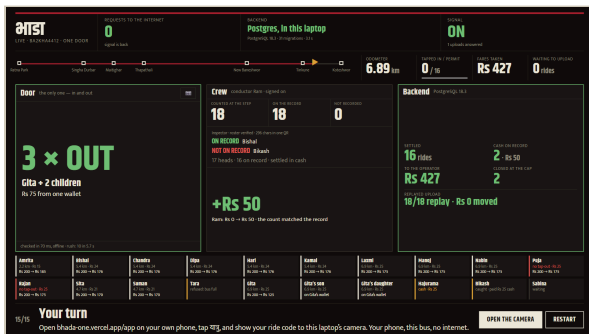
"This is the only moment that needed the internet — and the fares were already taken."



14 Send it all again

The whole upload replayed: every item "replay", Rs 0 moved.

"Nobody pays twice, however bad the connection."



15 Your turn

A judge opens bhada-one.vercel.app/app on their own phone and shows the ride code to the laptop camera.

"Your phone. This bus. No internet."

12. Questions judges ask, and the answers

QUESTION	ANSWER
What if people don't tap?	The counter at the step sees every body. The difference between bodies and rides + cash tickets is on the owner's dashboard for every trip, and the conductor only earns the bonus when it is under 10 %.
Rush hour?	0.2 s scanner reads, millisecond decisions, and a five-minute code so people tap once inside.
Why would conductors accept it?	Rs 50 per clean trip, paid automatically to their own wallet; no more arguing stages; cash is still allowed and recorded.
Why would owners pay for it?	For the first time they see every rupee against its kilometre and the number of people who rode unrecorded.
What does it cost per bus?	Day one: the phones the crew already has, plus a ₹1,500–2,500 USB scanner. A fixed Pi validator is about ₹4,500–7,000 in parts.
What if there's no internet?	Nothing at the door needs it. That is the whole design, and the demo shows it with the counter at zero.
What stops fraud by the bus itself?	A bus cannot bill anyone without that passenger's own signed tap; every fare is re-priced by the backend; the passenger's phone measures the ride as a witness; implausible rides are flagged.
What is proved and what is claimed?	The software is proved end to end — see section 13. It has not yet run on a moving bus: that is what the pilot is for.
What do you need?	A pilot on five R11 buses for four weeks, to measure tap rate against counted bodies and the fare accuracy on real roads.

13. What is proved, and what is live

PROOF	WHAT IT HOLDS	CHECKS
<code>proof</code>	The offline stage-fare handshake: accept, replay, edited amount, forged key, offline cap	all pass
<code>proof:meter</code>	Odometer accuracy on simulated roads, 400 random rides, the tariff promise over 21 stage pairs, receipts, taps, doors, the one-door rule, unclosed rides, the heartbeat frame, the pulled-plug rule, the clean-trip rule with the door count, cash tickets, the inspector's roster, family codes	115
<code>proof:reconcile</code>	Stage-fare settlement on real Postgres	all pass
<code>proof:legs</code>	Every migration on real Postgres: settlement, consent, replay, forgery, disputes, concessions, day-keys, overdraft, owners, admins, crew bonus, lost-phone moves, cash tickets and the door count, families, what the anon key can read	243
<code>proof:validator</code>	The Pi validator running the unchanged door offline: scanner, card reader, clock, RS-485, screens with every QR decoded from pixels, power tape, uplink, families	124

All five run before every deployment: the Vercel build is `npm run proof:all && npm run build`, so a failing proof never reaches the live site.

Live today

- Site: `bhada-one.vercel.app`, with `/app`, `/terminal`, `/device`, `/crew`, `/inspect`, `/operator`, `/admin` and `/demo`.
- Supabase: migrations 0001–0029, the sync function and the eSewa payments function.
- Migrations **0030** (cash tickets and door counts) and **0031** (the inspector's key register) ship with the next release, in the order database → function → site.

14. Known limits and the road ahead

Stated plainly, because a demo that hides these is how the earlier attempts failed

- **Not yet on a moving bus.** The $\pm 2\%$ accuracy is a result against a modelled GPS receiver. The trace recorder and `trace:replay` exist to score real roads; a four-week R11 pilot is the next step.
- **The door counter is software today.** The rule, the upload and the bonus are built and proved; the physical break-beam still has to be fitted and wired.
- **The Pi validator has not run on its parts.** Proved in software against the PN532 manual's frames, a simulated scanner and a decoded screen.
- **Cash riders do not count toward the interlock** (they never tap out); the door counter is what sees them.
- **Phones carry only R11's stops.** Eight more valley routes are on the backend; their stop lists need checking against DoTM permits.
- **A rider card is a serial, not a key.** The door holds the key that stands for it.
- **Meter keys are trusted on first use.** Production needs the owner to confirm a new bus's key from the dashboard.
- **Outside the software, and needed:** DoTM filing, NRB licensing for wallets, hardware procurement, conductor bonus agreements.

Next

1. Fit a break-beam counter to one bus; run the 4-week pilot on R11.
2. Build the first Pi validator from the parts list; measure scan-to-screen time.
3. Android app (Capacitor) for background GPS and the Play Store; SMS login.
4. Owner confirmation of new vehicle keys; stop lists for the other eight routes.

Appendix: addresses and commands

ADDRESS	WHAT IS THERE
bhada-one.vercel.app	Public site
/demo	The pitch demo
/app	Passenger ride card, account, conductor stage fares
/terminal?door=A	The door
/device	The meter console
/inspect	The inspector
/crew	Conductor sign-on
/operator,/admin	Owner dashboard, platform admin

COMMAND	WHAT IT DOES
npm install	Install (Node 22.13 or later)
npm run proof:all	All five proofs, no network
npm run build && npm run serve	Production build on http://localhost:4173 — use this for the demo
npm run dev	Development server
npm run validator:bench	The Pi validator on a laptop, no hardware
supabase db push	Apply migrations (release step 1)
supabase functions deploy sync --no-verify-jwt	Deploy the sync function (step 2)
git push origin master	Deploy the site through Vercel, after the proofs (step 3)

Bhada — भडा. Documentation generated from the repository on 28 September 2026. Screenshots are of the running software; the owner dashboard and account screens show sample data.